

Archivos

Un archivo es una estructura de datos consistente en una secuencia de elementos o componentes llamados registros, todos del mismo tipo, ya sea simple o estructurado. A diferencia de los arrays un archivo puede almacenarse en un dispositivo auxiliar disco, cintas, etc.. de forma que los datos obtenidos durante y después del procesamiento ni se pierdan.

Un archivo o fichero (file) es una colección de información (datos relacionados entre si) localizada o almacenada como una unidad en alguna parte de la computadora. Esta colección de datos sirve para entrada y salida a la computadora y se manejan con un programa. En contraste con los arrays y registros, el tamaño de esta colección no es fijo. Y esta limitado solo por la cantidad de memoria secundaria del disco o cinta disponible. Es decir los archivos son dinámicos: en un registro se deben identificar los campos, el número de elementos de un array y el número de elementos de una cadena , y son estructuras estáticas; en los archivos no se requieren tamaños predeterminados. Eso significa que se pueden hacer archivos de datos más grandes o más pequeños según se necesiten.

Los archivos no están limitados por la memoria de su computadora, donde están contenidas las estructuras de datos. En teoría se dispone de una unidad de disco de 20 MB, podría alojar un archivo de datos de igual tamaño, incluso con memoria de 128 KB o 640 KB, típicos de las computadoras actuales
Cada archivo es referenciado por un identificador (su nombre) .

Tipos de Archivos

Los elementos de un archivo pueden ser de cualquier tipo, simple o estructurado, excepto un tipo de archivo (file) o cualquier tipo estructurado con un componente tipo archivo.

Los principales tipos son:

1. archivo de entrada: una colección de datos localizado en un dispositivo de entrada
2. archivo de salida : un colección de información visualizada por la computadora
3. archivo de programa: un programa codificado en un lenguaje específico y localizado o almacenado en un dispositivo de almacenamiento

En Turbo Pascal existen 3 tipos de archivos los cuales son:

1. archivos tipeados (tipificados) o con tipo (**file of** .) (acceso aleatorio)
2. no tipeados (no tipificados) o sin tipo (file)
3. archivos secuenciales o archivos de texto

Tipos de acceso a un archivo

Existen dos modalidades para acceder a un archivo de datos : acceso secuencial y acceso directo o aleatorio. El acceso secuencial exige elemento a elemento, es

necesario una exploración secuencial comenzando desde el primer elemento. El acceso directo permite procesar o acceder a un elemento determinado haciendo una referencia directamente por su posición en el soporte de almacenamiento. Pascal estándar sólo admite el acceso secuencial, pero Turbo Pascal permite el acceso directo.

Los Archivos Tipeados (Con Tipo):

también llamados *archivos binarios*, contienen datos de tipo simple o estructurado, tales como integer, real, record, etc., excepto otro tipo de archivos. Los archivos con tipos están estructurados en elementos o registros (record) cuyo tipo puede ser cualquiera. A los elementos de estos archivos se accede directamente, al no situarse éstos en posiciones físicamente consecutivas, sino en posiciones lógicas. Esta es la razón por la cual se les denomina archivos de acceso aleatorio o directo. Los elementos de los archivos aleatorios son de igual tamaño y el término acceso directo significa que es posible acceder directamente a un elemento con solo especificar su posición. Declaración y asignación de archivos La declaración de un archivo con tipo se efectúa con la ayuda de las palabras reservadas file of.

El procedimiento de asignación es idéntico al utilizado anteriormente.

Ejemplo:

```
Type
datos = record
clave : integer;
nombre : string[30];
puesto : string[20];
sueldo : real;
estado : boolean;
{true activo,false baja lógica}
end;
Var
    archivo:file of datos;
Begin
    Assign(archivo,'empleado.dat');
```

Archivos Sin Tipo

Todos los archivos utilizados hasta ahora suponen algún tipo de estructura. Si no se conoce la estructura del registro se debe utilizar un archivo sin tipo. Los archivos sin tipo son canales de E/S de bajo nivel, principalmente utilizados para acceso directo a cualquier archivo de disco con independencia del tipo y estructura. Cualquier archivo de disco puede ser declarado como sin tipo. Turbo Pascal permite tratar un archivo como una serie de bloques sin necesidad de conocer el tipo de datos que contiene.

Archivos de Acceso Secuencial (Con Tipo)

Dependiendo de la manera en que se accedan los registros de un archivo, se le clasifica como secuencial o como directo. En el caso de los archivos de acceso secuencial, para tener acceso al registro localizado en la posición N, se deben haber accedido los N-1 registros previos, en un orden secuencial. Cuando se tienen pocos registros en un archivo, o que los registros son pequeños, la diferencia entre los tiempos de acceso de forma secuencial y directa puede no ser perceptible para el usuario; sin embargo, la diferencia viene a ser significativa cuando se manejan archivos con grandes cantidades de información. La forma de manejar los archivos de acceso secuencial es más sencilla en la mayoría de los lenguajes de programación, por lo que su estudio se antepone al de los archivos de acceso directo. El manejo secuencial de un archivo es recomendable cuando se deben procesar todos o la mayoría de los registros, como por ejemplo en los casos de una nómina de empleados o en la elaboración de reportes contables.

Tratamiento de Archivos desde Turbo Pascal

El Turbo Pascal permite manipular archivos y directorios en disco de modo similar al sistema operativo MS-DOS. Pudiéndose realizar las siguientes operaciones con archivos y directorios:

Erase(nomvararchivo)
Rename(nomvararchivo,'nombrenuevoarchivo')
Chdir (directorio)
Mkdir (directorio)
Rmdir (directorio)
Getdir(unidad,camino)
 Unidad = 0 unidad de arranque
 Unidad = 1 A:
 Unidad = 2 B:
 Unidad = 3 C:
Camino contiene el directorio actual.

Gestión de archivos

La siguiente tabla recopila todos los procedimientos y funciones estándar para el tratamiento y manipulación de archivos en turbo pascal

Todos tipos de archivos	Archivos de texto	Archivos sin tipo
Procedimientos		
Assign	Append	BlockRead
ChDir	Flush	BlockWrite
Close	Read	
Erase	ReadLn	
GetDir	SetTexBuf	
MkDir	Write	
Rename	WriteLn	
Reset		
Rewrite		
RmDir		
Funciones		
Eof	Eoln	
IOResult	SeekEof	
	SeekEoln	
<i>Procedimientos y funciones sobre cualquier tipo de variable excepto archivos de texto</i>		
FilePos		
FileSize		
Seek		
Truncate		

Assign

Éste procedimiento realiza la operación de asignar un archivo mediante una correspondencia entre una variable tipo archivo con un archivo externo situado en un disco.

Formato

Formato :assign (f,nombre)

f nombre interno del archivo por el que se conoce el archivo dentro del programa(por ejemplo, el utilizado en la declaración

nombre: nombre externo con el que se conoce el archivo por el sistema operativo (por ejemplo : 'a:program4.pas'); es una cadena que se define como una constante en el programa o bien una variable cuyo valor se lee interactivamente; puede ser una unidad:nombre.extensión o bien la ruta completa a un archivo.

Ejemplo :

Const

ruta ='c:\tp\lista.pas';

Var

ArchText : Text;

ArchBin : File;

Begin

assign (ArchText,ruta);

assign (ArchBin,'alumnos.dbf')

ChDir

Formato: **ChDir(s)**

Cambia el subdirectorio actual al especificado por la variable de cadena s.

Ejemplo (asignación del archivo lista.pas ubicado en c:\tp\lista.pas)

```
Var
  ArchText : Text;
Begin
  ChDir('C:\tp\');
  assign (ArchText,'lista.pas');
```

CLOSE

Éste procedimiento nos permite cerrar los archivos después que han sido utilizados, si los archivos no son cerrados se perderán los datos que se encuentran en la memoria auxiliar.

Formato : **Close (f)**

F : Variable de archivo.

Ejemplo:

```
Var
  ArchText : Text;
Begin
  assign (ArchText,'c:\tp\lista.pas');
  rewrite(ArchText);
  close(ArchText);
```

Erase

Éste procedimiento nos permite borrar un archivo, el archivo a borrar no debe estar abierto. Para borrar un archivo se debe realizar lo siguiente :

1. Asignar el archivo externo a una variable de archivo.
2. Llamar al procedimiento erase

Formato: **Erase(s)**

Borra (elimina) un archivo cuya ruta de acceso está especificada por s.

Ejemplo:

```
Var
  ArchText : Text;
Begin
  assign (ArchText,'c:\tp\lista.pas');
  erase(ArchText);
```

GetDir

Formato : **GetDir(i,s)**

Busca el nombre del subdirectorio actual. El primer parámetro i es un entero que representa la unidad : i=0 (unidad de disco actual, por defecto), i=1 (unidad A), i=2 (unidad B), etc. Después de la llamada, la variable de cadena s contiene el subdirectorio actual de esa unidad.

Ejemplo:

```
Var
  s:string[80];
Begin
  ChDir('c:\tp\');
  GetDir(0,s);
  WriteLn(s); {Resulta c:\tp}
```

MkDir

Formato:

MkDir(s)

Crea un nuevo subdirectorio de nombre s.

Ejemplo:

```
Begin
  MkDir('C:\nuevo\');
```

Rename

Éste procedimiento renombra (cambia el nombre) un archivo externo. Para renombrar un archivo se debe hacer lo siguiente :

1. Asignar un archivo a la variable archivo
2. Llamar al procedimiento rename

Formato: Rename(f,nombre_archivo)

f variable de archivo

nombre_archivo nuevo nombre del archivo

Ejemplo :

```
Var
  ArchText : Text;
Begin
  Assign (ArchText,'viejo.pas');
  Rename (ArchText,'nuevo.pas');
```

Reset

Este procedimiento abre un archivo existente para una operación de lectura. Si se intenta llamar a Reset y el archivo especificado no existe, se producirá un error de E/S (entrada/salida).

Formato : **Reset(f)**

f variable de tipo archivo

Ejemplo:

Var

ArchText : Text;

Begin

Assign (ArchText,'lista.pas');

Reset(ArchText);

Rewrite

Crea y abre un nuevo archivo. Si el archivo ya existe, Rewrite borra su contenido; en caso contrario, el archivo queda abierto para una operación de escritura.

Formato: **Rewrite(f)**

f variable de archivo

Ejemplo:

Var

ArchText : Text;

Begin

Assign (ArchText,'lista.pas');

Rewrite(ArchText);

RmDir

Elimina el subdirectorio de nombre s.

Formato **RmDir(s)**

Ejemplo

Begin

RmDir('C:\nuevo\');

Append

El procedimiento Append abre un archivo existente para añadir datos al final del mismo.

Formato: **Append(f)**

f variable de archivo de texto que debe haber sido asociada con un archivo externo por medio de Assign.

Si el archivo no existe, se produce un error; y si ya estaba abierto, primero se cierra y luego se reabre.

Ejemplo:

```
Var
  ArchText : Text;
Begin
  Assign (ArchText,'lista.pas');
  Rewrite(ArchText);
  WriteLn(ArchText,'Primera línea');
  Close(ArchText);
  Append(ArchText);
  WriteLn(ArchText,'Agrega esto al último');
  Close(ArchText)
end.
```

Flush

El procedimiento Flush vacía el *buffer* de un archivo de texto abierto para salida. Cuando un archivo de texto es abierto para escritura usando Rewrite o Append, la llamada a Flush llenará el *buffer* asociado al archivo. Esto garantiza que todos los caracteres escritos en el archivo en ese tiempo sean escritos en el archivo externo. Flush no afecta los archivos abiertos para entrada. Usando la directiva {\$I}, IOResult devuelve 0 si esta operación fue realizada con éxito de lo contrario retornará un código de error.

Formato: **Flush(f)**

f variable de archivo

Ejemplo:

```
Var
  ArchText : Text;
Begin
  Assign (ArchText,'lista.pas');
  Rewrite(ArchText);
  WriteLn(ArchText,'Primera línea');
  WriteLn(ArchText,'Segunda línea');
  Flush(ArchText);
  WriteLn(ArchText,'tercera línea');
  WriteLn(ArchText,'Agrega esto al último');
  Close(ArchText)
end.
```

Read

El procedimiento Read se utiliza para la lectura de datos situados en un archivo de tipo texto.

Formato: **Read(f,v1,v2,..)**

f variable de archivo de texto

v1,v2,... variable de tipo char, integer, real o string

Ejemplo:

Var

ArchText : Text;

datos : string[20];

Begin

Assign(ArchText,'lista.pas');

Reset(ArchText);

Read(ArchText,datos);

WriteLn(ArchText,datos);

Close(ArchText)

End.

ReadLn

EL procedimiento ReadLn se utiliza para la lectura de datos situados en un archivo de tipo texto. A diferencia de Read, ReadLn salta al principio de la siguiente línea del archivo. Este salto de línea se produce cuando se han asignado valores a la lista de variables del procedimiento; en caso contrario, el procedimiento hace caso omiso del control de línea y sigue asignando información.

Formato: **ReadLn(f,v1,v2,..)**

f variable de archivo de texto

v1,v2,... variable de tipo char, integer,
real o string

Ejemplo:

Var

ArchText : Text;

datos : string[20];

Begin

Assign (ArchText,'lista.pas');

Reset(ArchText);

ReadLn(ArchText,datos);

WriteLn(ArchText,datos);

Close(ArchText)

End.

SetTextBuf

El procedimiento SetTextBuf asigna un *buffer* de entrada/salida a un archivo de texto. SetTextBuf podría nunca ser usado en la apertura de un archivo, aunque éste puede ser llamado inmediatamente después de Reset, Rewrite, y Append. Si se invoca a SetTextBuf en una apertura de archivo las operaciones de entrada/salida son tomadas por éste procedimiento, y se pueden perder datos como resultado de el cambio de *buffer*.

formato : **SetTextBuf(f,buffer)**

Ejemplo:

Var

ArchText: Text;

Ch: Char;

buffer: array[1..4096] of Char; { buffer de 4K}

Begin

Assign(ArchText,'memoria.txt');

{Se utiliza un buffer grande para lecturas más rápidas}

SetTextBuf(ArchText,buffer);

Reset(ArchText); {Visualiza el archivo en la pantalla}

while not Eof(ArchText) do

Begin

Read(ArchText,Ch);

Write(Ch)

End;

Close(ArchText)

end.

Write

EL procedimiento Write sirve para escribir datos en un archivo.

Formato: **Write(f,v1,v2,...)**

f variable de tipo archivo

v1,v2,... variables de tipo de datos

Ejemplo:

Var

ArchText : Text;

Begin

Assign (ArchText,'nuevo.txt');

Rewrite(ArchText);

Write(ArchText,'Está es la primera línea.');

Write(ArchText,'Fin del texto.');

Close(ArchText)

End.

El contenido del archivo será : Está es la primera línea. Fin del texto.

WriteLn

EL procedimiento WriteLn sirve para escribir datos en un archivo. A diferencia de Write, WriteLn incluye un salto de línea para separar el texto.

Formato: **WriteLn(f,v1,v2,...)**

f variable de tipo archivo

v1,v2,... variables de tipo de datos

Ejemplo:

Var

ArchText : Text;

Begin

Assign (ArchText,'nuevo.txt');

Rewrite(ArchText);

WriteLn(ArchText,'Está es la primera línea.');

WriteLn(ArchText,'Fin del texto.');

Close(ArchText)

end.

El contenido del archivo será :

Está es la primera línea. Fin del texto.

BlockRead

Transfiere un bloque de datos de un archivo sin tipo hacia un *buffer*.

Formato: **BlockRead(arch,buffer,bloques,resul)**

arch archivo sin tipo

buffer variable de cualquier tipo de longitud suficiente para acogerlos datos transferidos

bloques expresión que corresponde al número de bloques de 128 bytes a transferir.

resul parámetro opcional que indica el número de bloques que Se leyeron o escribieron realmente.

BlockWrite

Transfiere un buffer de datos hacia un archivo sin tipo.

Formato: **BlockWrite(arch,buffer,bloques,resul)**

Arch: archivo sin tipo

buffer variable de cualquier tipo de longitud suficiente para acoger los datos transferidos

bloques expresión que corresponde al número de bloques de 128 bytes a transferir.

Resul: parámetro opcional que indica el número de bloques que se leyeron o escribieron realmente.

Ejemplo:

Program CopiadeArchivos;

Var

fuente,destino : file; {archivo sin tipo}

buffer : array[1..512] of byte;

leidos : integer;

Begin

Assign(fuente,'original.txt');

Assign(destino,'copia.txt');

Reset(fuente,1);

Rewrite(destino,1);

BlockRead(fuente,buffer,SizeOf(buffer),leidos);

while leidos>0 do

Begin

BlockWrite(destino,buffer,SizeOf(buffer),leidos);

BlockRead(fuente,buffer,SizeOf(buffer),leidos)

end

close(fuente);

close(destino)

end.

Eof

La función eof(*end of file*), fin de archivo, devuelve el estado de un archivo. Es una función de tipo lógico que indica si el fin de archivo se ha encontrado; devuelve true si se encontró, false en caso contrario.

Formato: **Eof(f)**

f variable de archivo

Ejemplo:

Var

ArchText: Text;

Ch: Char;

Begin

Assign(ArchText,'memoria.txt');

Reset(ArchText); {Visualiza el archivo en la pantalla}

while not Eof(ArchText) do

Begin

Read(ArchText,Ch);

Write(Ch)

end;

```
Close(ArchText)
end.
```

Eoln

La función Eoln devuelve el estado de fin de línea de un archivo. Es una función de tipo lógico. Devuelve true si la posición actual del archivo (puntero) esta en la marca de fin de línea, o bien si Eof(f) es true, el caso contrario devuelve false.

Formato: **Eoln(f)**
f variable de archivo de texto

```
Var
car :char;
ArchText:text;
Begin
Clrscr;
Assign(ArchText,'nuevo.txt');
Reset(ArchText);
{muestra sola la primera línea}
While not(Eoln(ArchText)) do
    Begin
        Read(ArchText,car);
        Write(car)
    end
end.
```

SeekEof

Retorna el estado de fin de archivo. Es una función de tipo lógico. Sólo para archivos de texto. Si la posición actual del puntero de archivo se encuentra en la marca de fin de archivo devuelve true, de lo contrario retorna false. SeekEof es una función que avanza siempre al siguiente carácter, ignorando por completo los espacios en blanco.

Formato: **SeekEof(f)**
f variable de archivo de texto

Ejemplo:

```
Var
car :char;
ArchText:text;
Begin
Clrscr;
Assign(ArchText,'nuevo.txt');
Reset(ArchText);
{muestra el contenido del archivo}
```

```

While not(SeekEof(ArchText)) do
  Begin
    Read(ArchText,car);
    Write(car)
  End
end.

```

SeekEoln

Retorna el estado de fin de línea. Es una función de tipo lógico. Sólo para archivos de texto. Si la posición actual del puntero de archivo se encuentra en la marca de fin de línea devuelve true, de lo contrario retorna false. SeekEoln es una función que avanza siempre al siguiente carácter, ignorando por completo los espacios en blanco.

Formato: **SeekEoln(f)**
 f variable de archivo de texto

Ejemplo:

```

Var
  car :char;
  ArchText:text;
Begin
  Clrscr;
  Assign(ArchText,'nuevo.txt');
  Reset(ArchText);
  {muestra el contenido del archivo}
  While not(SeekEoln(ArchText)) do
    Begin
      Read(ArchText,car);
      Write(car)
    end;
  Close(ArchText)
end.

```

IOResult

Es una función que devuelve un número entero que indica el código del error de la última operación de E/S; si el valor de retorno es cero, esto nos indica que la operación se ha desarrollado sin problema.

Formato: **IOResult**

Ejemplo:

```

Var
  ArchText:text;
  Nerror :integer;

```

```

Begin
Clrscr;
Assign(ArchText,'nuevo.txt');
{SI-}
Reset(ArchText);
Nerror:=IOResult;
{SI+}
if Nerror<>0 then
WriteLn('Código de error # ',Nerror)
else
Close(ArchText)
end.

```

FilePos

Esta función devuelve la posición actual del archivo (número de registro), en forma de un entero largo (longint).

Formato: **FilePos (f)**

f variable de tipo archivo

Notas :

1. Devuelve 0 si esta al principio del archivo.
2. Es igual a FileSize(f) si el puntero de posición del archivo esta al final del archivo.

FileSize

Esta función devuelve el tamaño actual del archivo(número de registros existentes en el archivo). Si el archivo esta vacío devuelve cero.

Formato: **FileSize(f)**

f variable de tipo archivo

Seek

Sitúa el puntero de posición del archivo en el número de registro correspondiente.

Formato: **Seek(f,numreg)**

f nombre de archivo (no de tipo text) numreg número de posición del registro, el primer registro es cero; si numreg es menor que 0 o mayor que n (para un archivo de n registros), se producirán resultados impredecibles.

Ejemplo:

```

Seek(empleados,FileSize(empleados));
{está sentencia posiciona el puntero al final del archivo}

```

Truncate

Esta función trunca (corta, mutila) un archivo a partir de la posición actual del puntero del archivo.

Formato: **Truncate(f)**

f variable de tipo archivo

Si Truncate se ha ejecutado con éxito IOResult tendrá un valor de cero. El archivo debe estar abierto. Truncate no trabaja en archivos de texto.

Ejemplo:

Var

f: file of integer;

i,j: integer;

Begin

Assign(f,'prueba.int');

Rewrite(f);

For i := 1 to 5 do

Write(f,i);

Writeln('Archivo antes de ser truncado :');

Reset(f);

while not Eof(f) do

Begin

Read(f,i);

Writeln(i)

end;

Reset(f);

For i := 1 to 3 do

Read(f,j); { lectura de 3 registros }

Truncate(f); { Trunca el archivo a partir de aquí }

Writeln;

Writeln('Archivo después de ser truncado:');

Reset(f);

while not Eof(f) do

Begin

Read(f,i);

Writeln(i)

end;

Close(f);

end.